Вот текущая версия lstm^

```
inline float sigmoid(float x) {
    return 1.0f / (1.0f + exp(-x));
}
void atomic_add_float(global volatile float* source, const float val) {
    union { unsigned int intVal; float floatVal; } next, expected, current;
    current.floatVal = *source;
    do {
        expected.floatVal = current.floatVal;
        next.floatVal = expected.floatVal + val;
        current.intVal = atomic_cmpxchg((global volatile unsigned int*)source, expected.intVal,
next.intVal);
    } while (current.intVal != expected.intVal);
}

/**
 * Монолитное ядро прямого прохода LSTM.
 * Сетка запуска (Global Work Size): [BATCH_SIZE]
 */


kernel void lstm_forward(
    global const float* X,
    global const float* W,
    global const float* U,
    global const float* bias,
    global float* H_out,
    global float* workspace,
    global float* c_curr_buf,
    global float* h_curr_buf,
    const int batch_size,
    const int seq_len,
    const int in_features,
    const int hidden_size
) {
    // get_global_id(0) отвечает за BATCH_SIZE [0 .. batch_size-1]
    // get_global_id(1) отвечает за HIDDEN_SIZE [0 .. hidden_size-1]
    int b = get_global_id(0);
    int hid = get_global_id(1);

    if (b >= batch_size || hid >= hidden_size) return;

    // Ссылки на ячейку текущего потока в глобальной памяти
    global float* c_curr_ptr = c_curr_buf + (b * hidden_size);
    global float* h_curr_ptr = h_curr_buf + (b * hidden_size);

    // Шаг 1: Инициализация (только один раз для каждого элемента скрытого состояния)
    c_curr_ptr[hid] = 0.0f;
    h_curr_ptr[hid] = 0.0f;

    // Синхронизируем инициализацию перед циклом по времени
    barrier(CLK_GLOBAL_MEM_FENCE);
```

```
    int x_offset;
    int ws_offset;
    int h_out_offset;

    // Шаг 2: Основной цикл по временной оси (T)
    for (int t = 0; t < seq_len; t++) {
        x_offset = b * (seq_len * in_features) + t * in_features;
        ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;

        // Каждый поток обсчитывает свои 4 гейта для индекса 'hid'
        for (int g = 0; g < 4; g++) {
            // Индекс гейта в плоском массиве весов: [g * hidden_size + hid]
            int gate_idx = g * hidden_size + hid;
            float val = bias[gate_idx];

            // 1. Умножение на входные веса: X_t * W
            for (int i = 0; i < in_features; i++) {
                val += X[x_offset + i] * W[i * (4 * hidden_size) + gate_idx];
            }

            // 2. Умножение на рекуррентные веса: H_{t-1} * U
            // Внимание: здесь мы последовательно читаем h_curr_ptr,
            // который был синхронизирован барьером в конце предыдущей итерации t
            for (int h = 0; h < hidden_size; h++) {
                val += h_curr_ptr[h] * U[h * (4 * hidden_size) + gate_idx];
            }

            // Запись в workspace
            workspace[ws_offset + gate_idx] = val;
        }

        // Ждем, пока ВСЕ потоки текущего батча посчитают и запишут свои гейты в workspace
        barrier(CLK_GLOBAL_MEM_FENCE);

        // Шаг 3: Расчет активаций и обновление состояний (без цикла по h!)
        float raw_i = workspace[ws_offset + 0 * hidden_size + hid];
        float raw_f = workspace[ws_offset + 1 * hidden_size + hid];
        float raw_g = workspace[ws_offset + 2 * hidden_size + hid];
        float raw_o = workspace[ws_offset + 3 * hidden_size + hid];

        float i_gate = sigmoid(raw_i);
        float f_gate = sigmoid(raw_f);
        float g_gate = tanh(raw_g);
        float o_gate = sigmoid(raw_o);

        // Обновляем память ячейки c_t и скрытого состояния h_t
        float c_val = f_gate * c_curr_ptr[hid] + i_gate * g_gate;
        c_curr_ptr[hid] = c_val;

        float h_val = o_gate * tanh(c_val);
```

```
        h_curr_ptr[hid] = h_val;

        // Пишем в глобальный выходной тензор
        H_out[h_out_offset + hid] = h_val;

        // ВАЖНО: Барьер в конце шага. Гарантирует, что на следующей итерации t+1
        // при чтении `h_curr_ptr[h]` все потоки увидят обновленные h_val.
        barrier(CLK_GLOBAL_MEM_FENCE);
    }
}


/**
 * Монолитное ядро обратного прохода LSTM (BPTT).
 * Сетка запуска (Global Work Size): [BATCH_SIZE]
 */


kernel void lstm_backward(
    global const float* X,
    global const float* W,
    global const float* U,
    global const float* H_out,
    global const float* workspace,
    global const float* grad_H_out,
    global float* grad_X,
    global float* grad_W,
    global float* grad_U,
    global float* dh_next_buf,
    global float* dc_next_buf,
    global float* c_accum_buf,
    global float* grad_gates_buf,
    const int batch_size,
    const int seq_len,
    const int in_features,
    const int hidden_size
) {
    int b = get_global_id(0);
    int hid = get_global_id(1);

    if (b >= batch_size || hid >= hidden_size) return;

    global float* dh_next = dh_next_buf + (b * hidden_size);
    global float* dc_next = dc_next_buf + (b * hidden_size);
    global float* c_accum = c_accum_buf + (b * hidden_size);
    global float* grad_gates = grad_gates_buf + (b * 4 * hidden_size);

    // Инициализация градиентов "из будущего"
    dh_next[hid] = 0.0f;
    dc_next[hid] = 0.0f;
    c_accum[hid] = 0.0f;
```

```
    barrier(CLK_GLOBAL_MEM_FENCE);

    // Шаг 1: Прямая регенерация цепочки состояний C (параллельно по hid)
    for (int t = 0; t < seq_len; t++) {
        int ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        float i_gate = sigmoid(workspace[ws_offset + 0 * hidden_size + hid]);
        float f_gate = sigmoid(workspace[ws_offset + 1 * hidden_size + hid]);
        float g_gate = tanh(workspace[ws_offset + 2 * hidden_size + hid]);

        c_accum[hid] = f_gate * c_accum[hid] + i_gate * g_gate;
    }

    barrier(CLK_GLOBAL_MEM_FENCE);

    // Шаг 2: Обратный проход во времени (BPTT)
    for (int t = seq_len - 1; t >= 0; t--) {
        int ws_offset = t * (batch_size * 4 * hidden_size) + b * (4 * hidden_size);
        int h_out_offset = b * (seq_len * hidden_size) + t * hidden_size;
        int x_offset = b * (seq_len * in_features) + t * in_features;

        float i_gate = sigmoid(workspace[ws_offset + 0 * hidden_size + hid]);
        float f_gate = sigmoid(workspace[ws_offset + 1 * hidden_size + hid]);
        float g_gate = tanh(workspace[ws_offset + 2 * hidden_size + hid]);

        // Восстановление c_{t-1}
        float c_prev_val = (f_gate != 0.0f) ? (c_accum[hid] - i_gate * g_gate) / f_gate : 0.0f;

        float dh_curr = grad_H_out[h_out_offset + hid] + dh_next[hid];
        float raw_o = workspace[ws_offset + 3 * hidden_size + hid];
        float o_gate = sigmoid(raw_o);
        float tanh_c = tanh(c_accum[hid]);

        // Локальные градиенты по гейтам
        float d_raw_o = dh_curr * tanh_c * (o_gate * (1.0f - o_gate));
        float dc_curr = dh_curr * o_gate * (1.0f - tanh_c * tanh_c) + dc_next[hid];

        float d_raw_f = dc_curr * c_prev_val * (f_gate * (1.0f - f_gate));
        float d_raw_i = dc_curr * g_gate * (i_gate * (1.0f - i_gate));
        float d_raw_g = dc_curr * i_gate * (1.0f - g_gate * g_gate);

        // Пишем в глобальный буфер гейтов
        grad_gates[0 * hidden_size + hid] = d_raw_i;
        grad_gates[1 * hidden_size + hid] = d_raw_f;
        grad_gates[2 * hidden_size + hid] = d_raw_g;
        grad_gates[3 * hidden_size + hid] = d_raw_o;

        // Обновляем значения для следующего шага назад (t-1)
        dc_next[hid] = dc_curr * f_gate;
        c_accum[hid] = c_prev_val;

        // Барьер: все гейты текущего шага времени `t` должны быть полностью вычислены
        barrier(CLK_GLOBAL_MEM_FENCE);
```

```
        // Вычисляем градиент по входу dL/dX_t (накапливаем вклад от нашего hid атомарно)
        int x_grad_offset = b * (seq_len * in_features) + t * in_features;
        for (int i = 0; i < in_features; i++) {
            float sum_x_contribution = 0.0f;
            for (int g = 0; g < 4; g++) {
                int gate_idx = g * hidden_size + hid;
                sum_x_contribution += grad_gates[gate_idx] * W[i * (4 * hidden_size) + gate_idx];
            }
            // Атомарно добавляем вклад этого скрытого элемента в общий вход
            atomic_add_float(&grad_X[x_grad_offset + i], sum_x_contribution);
        }

        // Вычисляем dh_next для шага (t-1). Каждый поток `hid` собирает данные из всего массива гейтов.
        float sum_h = 0.0f;
        for (int g = 0; g < 4; g++) {
            for (int h_idx = 0; h_idx < hidden_size; h_idx++) {
                int gate_idx = g * hidden_size + h_idx;
                // Вклад рекуррентных весов
                if (h_idx == hid) {
                    // Для оптимизации: здесь логика зависит от вашей структуры матрицы U.
                }
                // Нам нужно умножить вектор grad_gates на строку матрицы U^T.
                // grad_gates имеет размер [4 * hidden_size].
                // Поток `hid` должен собрать: sum_{g=0..3} sum_{h=0..hidden-1} grad_gates[g, h] * U[hid, g, h]
                sum_h += grad_gates[g * hidden_size + h_idx] * U[hid * (4 * hidden_size) + (g * hidden_size + h_idx)];
            }
        }

        // Синхронизируем вычисление sum_h перед записью в dh_next
        barrier(CLK_GLOBAL_MEM_FENCE);
        dh_next[hid] = sum_h;

        // НАКОПЛЕНИЕ ГРАДИЕНТОВ ПО ВЕСАМ (W, U) с использованием атомиков
        int h_prev_offset = (t > 0) ? (b * (seq_len * hidden_size) + (t - 1) * hidden_size) : -1;

        for (int g = 0; g < 4; g++) {
            int gate_idx = g * hidden_size + hid;
            float g_grad = grad_gates[gate_idx];

            // Градиент весов W
            for (int i = 0; i < in_features; i++) {
                float w_delta = X[x_offset + i] * g_grad;
                atomic_add_float(&grad_W[i * (4 * hidden_size) + gate_idx], w_delta);
            }

            // Градиент весов U
            if (h_prev_offset != -1) {
                for (int h = 0; h < hidden_size; h++) {
```

```
                float u_delta = H_out[h_prev_offset + h] * g_grad;
                atomic_add_float(&grad_U[h * (4 * hidden_size) + gate_idx], u_delta);
            }
        }
    }

    // Финальный барьер шага `t`, чтобы подготовить `dh_next` и `c_accum` к шагу `t-1`
    barrier(CLK_GLOBAL_MEM_FENCE);
  }
}
```

Но мне кажеться что проблема в Linear.
Я провел тестирование этого слоя следующим образом:

```
    static void Main(string[] args)
    {
        Console.WriteLine("=== ТЕСТИРОВАНИЕ ОБУЧАЕМОГО СЛОЯ LINEAR (OPENCL JIT) ===");

        try
        {
            // 1. Инициализация OpenCL
            using var computeContext = new ComputeContext(preferAccelerator: true);
            using var queue = computeContext.Context.CreateCommandQueue(computeContext.Device);
            DeviceQueueRegistry.RegisterCurrent(queue);

            Console.WriteLine($"Выбран девайс: {computeContext.Device.Name}");

            // Геометрия: Батч = 2 примера, Входных фичей = 3, Выходных фичей = 4
            int batch = 2; int inFeatures = 3; int outFeatures = 4;

            float[] inputData = new float[] {
                1.0f, 2.0f, 3.0f,  // Пример 1
                4.0f, 5.0f, 6.0f   // Пример 2
            };
            float[] targetData = new float[] {
                0.0f, 1.0f, 0.0f, 0.0f,  // Желаемый выход для Примера 1
                1.0f, 0.0f, 0.0f, 0.0f   // Желаемый выход для Примера 2
            };

            // Инициализация тензоров данных
            using var xTensor = new Tensor(inputData, new[] { batch, 1, inFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad: true);
            using var targetTensor = new Tensor(targetData, new[] { batch, 1, outFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad: false);

            var nnClGlob = new NnClGlob(computeContext);

            // 2. Создание нашего нового слоя Linear!
            using var linearLayer = new Linear(computeContext, nnClGlob, inFeatures, outFeatures);

            // Передаем параметры слоя в оптимизатор SGD
            var optimizer = new SGD(linearLayer.GetParameters(), learningRate: 0.2f, computeContext);

            Console.WriteLine("\n--- Выполнение шага обучения слоя Linear ---");
```

```
            for (int epoch = 0; epoch < 10; epoch++)
            {
                Console.WriteLine($"эпоха {epoch+1}!");
               // GradientStorage.Clear();
                linearLayer.ZeroGrad(); // Сбрасываем старые буферы градиентов параметров

                // А. Прямой проход слоя (BMM + JIT Bias)
                Tensor pred = linearLayer.Forward(xTensor);
                Console.WriteLine($"[Forward] Выходной тензор успешно рассчитан. Форма: [{string.Join(", ", pred.Shape)}]");

                // Б. Расчет величины ошибки
                Tensor loss = nnClGlob.MseLoss(pred, targetTensor);
                float[] lossCpu = HostReturner.ToCpu(loss);
                Console.WriteLine($"[Loss] Текущая ошибка слоя: {lossCpu[0]:F6}");

                // В. ЗАПУСК ОБРАТНОГО ПРОХОДА (Autograd)
                // Градиент пойдет: MSE -> LocalOp (дельта по bias) -> BmmOp -> Веса Weight -> Входной тензор X
                AutoGrad.Backward(loss, computeContext.Context);
                Console.WriteLine("[Backward] Каскадный обратный проход графа успешно завершен!");

                // Г. Проверка сцепки буферов градиентов на GPU во VRAM
                Console.WriteLine("\nАнализ буферов градиентов после сцепки:");
                Console.WriteLine($"  -> Градиент для смещения Bias создан: {linearLayer.Bias.Grad != null}");
                Console.WriteLine($"  -> Градиент для матрицы весов Weight создан: {linearLayer.Weight.Grad != null}");
                Console.WriteLine($"  -> Градиент дотек до самого первого входа X: {xTensor.Grad != null}");

                if (linearLayer.Bias.Grad != null && linearLayer.Weight.Grad != null)
                {
                    float[] biasGrad = HostReturner.ToCpuGrad(linearLayer.Bias);
                    float[] weightGrad = HostReturner.ToCpuGrad(linearLayer.Weight);

                    Console.ForegroundColor = ConsoleColor.Green;
                    Console.WriteLine($"\n[Успех] Градиент Bias (все 4 эл.): [{string.Join(", ", biasGrad.Select(x => x.ToString("F4")))}]");
                    Console.WriteLine($"        Градиент весов Weight (первые 4): {weightGrad[0]:F5}, {weightGrad[1]:F5}, {weightGrad[2]:F5}, {weightGrad[3]:F5}");
                    Console.ResetColor();

                    // Д. Физическое обновление параметров во VRAM
                    optimizer.Step();
                    Console.WriteLine("\n[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.");
                }
                else
                {
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.WriteLine("\n[Ошибка] Градиенты застряли на стыке слоев!");
                    Console.ResetColor();
                }
            }
                Console.WriteLine("\n=== ТЕСТ ОБУЧАЕМОГО СЛОЯ LINEAR С УСПЕХОМ ЗАВЕРШЕН! ===");
```

```
        }
        catch (Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"\n[КРИТИЧЕСКАЯ ОШИБКА]: {ex.Message}");
            Console.ResetColor();
        }
        finally
        {
            DeviceQueueRegistry.Clear();
        }

        Console.ReadLine();
    }
```

И получил следующие результаты:
максимальные размеры локальной группы для устройства на основе варпа:X=256,Y=4
Выбран девайс: NVIDIA GeForce RTX 2060
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-430e4c.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-e53138.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-63046e.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-636bcd.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-1288d6.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-5841c8.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-7d3d18.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-30fa9d.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-bf92dd.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-c23379.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-fb6d4a.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-04cc1f.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-2c71ca.d

--- Выполнение шага обучения слоя Linear ---
эпоха 1!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-a0a180.d
[Loss] Текущая ошибка слоя: 26,565073
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-54314c.d
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-905f0a.d
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [1,3744, -3,4366, -0,5149, -3,0895]
      Градиент весов Weight (первые 4): 4,43884, -10,11251, -1,98996, -9,47381
Could not open file: C:\Users\╨Ф╨╛╨╗ ╨┐╤B╤A╨┐ ╨╡╨╜\AppData\Local\Temp\dep-2d223c.d

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 2!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 343,153900

[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [-5,1418, 11,8209, 2,2697, 11,0288]
      Градиент весов Weight (первые 4): -15,77562, 37,17697, 6,66240, 34,30213

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 3!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 4460,111816
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [18,3640, -43,1732, -7,7899, -39,8773]
      Градиент весов Weight (первые 4): 57,10831, -133,38185, -24,51575, -123,56159

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 4!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 57995,136719
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [-66,3911, 155,1616, 28,4680, 143,6967]
      Градиент весов Weight (первые 4): -205,72346, 481,62955, 87,93535, 445,69479

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 5!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 754137,750000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [239,2465, -560,0176, -102,2959, -518,2734]
      Градиент весов Weight (первые 4): 742,04810, -1736,15222, -317,54672, -1607,06995

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 6!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 9806426,000000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [-862,8868, 2018,9661, 369,2277, 1868,8191]
      Градиент весов Weight (первые 4): -2675,65918, 6261,22168, 1144,65613, 5795,26807

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 7!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 127517808,000000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [3111,4524, -7280,9209, -1331,1187, -6739,1182]
      Градиент весов Weight (первые 4): 9648,70703, -22577,62109, -4128,08105, -20897,83398

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 8!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 1658177024,000000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [-11220,1514, 26254,8223, 4800,3770, 24301,4141]
      Градиент весов Weight (первые 4): -34793,41016, 81416,25000, 14885,61719, 75358,43750

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 9!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 21562097664,000000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
  -> Градиент для смещения Bias создан: True
  -> Градиент для матрицы весов Weight создан: True
  -> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [40460,1328, -94676,2578, -17310,0293, -87631,8516]
Градиент весов Weight (первые 4): 125466,46094, -293589,43750, -53678,44531, -271745,12500

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.
эпоха 10!
[Forward] Выходной тензор успешно рассчитан. Форма: [2, 1, 4]
[Loss] Текущая ошибка слоя: 280382636032,000000
[Backward] Каскадный обратный проход графа успешно завершен!

Анализ буферов градиентов после сцепки:
-> Градиент для смещения Bias создан: True
-> Градиент для матрицы весов Weight создан: True
-> Градиент дотек до самого первого входа X: True

[Успех] Градиент Bias (все 4 эл.): [-145900,8281, 341405,5938, 62420,8711, 316003,5312]
Градиент весов Weight (первые 4): -452436,40625, 1058694,50000, 193566,03125, 979922,81250

[SGD] Шаг оптимизации успешно применен. Веса слоя Linear обновлены на GPU.

=== ТЕСТ ОБУЧАЕМОГО СЛОЯ LINEAR С УСПЕХОМ ЗАВЕРШЕН! ===

Сеть не учиться , а деградирует при этом ошибка растет весьма сильно!